Zephyr Project

IT-Schulungen.com
Technology & Transformation

Einrichtung und Kursstart

Workspace · Checkout · west · Zephyr · native_sim

1.01

Arbeitsplatz vorbereiten

HOST + VENV

```
$ sudo apt install git cmake ninja-build \
gperf ccache device-tree-compiler \
python3-venv gcc gcc-multilib g++-multilib

$ mkdir training-zephyr-ws && cd $_
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install --upgrade pip west
```

01 Host-Pakete

- Linux, WSL 2 oder VM
- Build-Werkzeuge und Multilib installieren

02 Python-Umgebung

- venv isoliert Abhängigkeiten
- west wird im venv installiert

Ergebnis: Shell, Compiler, Python-Umgebung und west sind einsatzbereit.

Unterlagen auschecken und Workspace initialisieren

CHECKOUT + WEST

```
$ cd ~/training-zephyr-ws
$ git clone <Workshop-Repo> training
$ west init -l training
$ west update --narrow
$ west zephyr-export
```

01 Manifest-Repo

- training enthält west.yml
- Folien, Übungen und Exkurse

02 west-Workspace

- west init registriert das Manifest
- west update holt Zephyr und Module

Ergebnis: training, zephyr, modules, bootloader und tools liegen im gemeinsamen Workspace.

Zephyr installieren und testen

ZEPHYR + ERSTTEST

```
$ pip install -r zephyr/scripts/requirements.txt
$ west sdk install --toolchains arm-zephyr-eabi
```

01 Toolchain

- SDK 1.0.x für Zephyr v4.4.2
- native_sim kann den Host-Compiler nutzen

02 Erfolgskriterium

- Build endet fehlerfrei
- Hello World läuft als Linux-Prozess

Ergebnis: Zephyr-Pakete und Toolchain sind aktiv; native_sim läuft als Hostprozess.

Ziele und Arbeitsergebnisse von Tag 1

VERSTEHEN

Zephyr einordnen

RTOS und Architektur

Workspace und west

Kconfig, Devicetree und CMake

ANWENDEN

Zyklus beherrschen

Konfigurieren und bauen

Als Linux-Prozess starten

Ausgaben beobachten

DIAGNOSTIZIEREN

Probleme eingrenzen

Fehlermeldungen lesen

Build-Artefakte prüfen

Sollzustand bestätigen

Das Tagesergebnis ist eine nachvollziehbare Zephyr-Anwendung für native_sim.

So arbeiten wir im Kurs

KURZINPUT

Konzept verstehen

Begriffe und Modell

Kernidee visualisieren

Fragen klären

DEMONSTRATION

Vorgehen beobachten

Befehl und Ergebnis

Entscheidungen erklären

Fehler sichtbar machen

PRAXIS

Selbst anwenden

Geführt beginnen

Eigenständig erweitern

Gemeinsam reflektieren

Kurze Schleifen halten Theorie, Demonstration und eigene Anwendung eng zusammen.

Unsere Arbeitsumgebung

START-CHECK

```
$ uname -s
Linux
$ west topdir
~/training-zephyr-ws
$ west build -b native_sim ...
```

01 Linux-Basis

- Linux nativ, WSL 2 oder VM
- Einheitliche Shell und Host-Tools

02 Zephyr-Workspace

- west verwaltet Workspace und Build
- Die Toolchain erzeugt ein Host-Programm

Für Tag 1 nutzen wir die gemeinsam eingerichtete Linux-Umgebung und den geprüften Workspace.

Warum wir mit native_sim beginnen

NATIVE_SIM

```
schneller Build
Start ohne Flashen
Logs direkt im Terminal
leicht automatisierbar
```

01 Frühes Feedback

- Anwendung und Zephyr laufen als Linux-Prozess.
- Fehler werden schnell reproduzierbar.

02 Bewusste Grenze

- Kein reales MCU-Timing.
- Keine echte Peripherie oder elektrische Effekte.

native_sim beschleunigt Lernen und Tests; Hardwarevalidierung bleibt später notwendig.

Gemeinsamer Startpunkt

EINSTIEGSFRAGE

Welche Erfahrung bringen Sie mit?

Was soll nach fünf Tagen leichter sein?

Welche technische Hürde erwarten Sie?

01 Jetzt klären

Lernziele, Rollen und offene Fragen

02 Danach bereit

Arbeitsumgebung ist bekannt und der Tagesablauf ist transparent.

Ergebnis von 1.01: gemeinsames Zielbild, klare Arbeitsweise und Fokus auf native_sim.